1. **Unificación de Lenguajes Backend:** En la segunda parte de tu documento preliminar se menciona C# .NET en un par de casos de uso, mientras que tus apuntes de clase imponen explícitamente **Spring Boot (Java), NestJS y PHP**, permitiendo agregar un microservicio adicional en otro lenguaje (como **Python** para IA). En esta propuesta unificamos todo con **Java (Spring Boot), TypeScript (NestJS), PHP y Python**.
2. 
3. **Unificación de Patrones de Arquitectura de Código:** Tus apuntes manuscritos exigen explícitamente **DDD (Domain-Driven Design), Arquitectura Hexagonal y Clean Architecture**, mientras que el documento mencionaba Arquitectura por Capas en lugar de DDD. Aquí integraremos **DDD táctico y estratégico**, **Hexagonal** y **Clean Architecture** para cumplir al 100% con tu hoja de condiciones.
4. 
5. **Flujo de n8n y Facturación:** Tus apuntes piden que la automatización en **n8n** tome pedidos/solicitudes por WhatsApp, los recepcione en el ERP y pase a generar la factura o nota de venta, además del reporte de siniestros. Integraremos ambos procesos dentro del dominio asegurador (venta/renovación de póliza con emisión de factura y atención de siniestro vial por WhatsApp).
6. 

### Propuesta Integral del Proyecto: "AegisCore AI"

- **Nombre Comercial: AegisCore AI** (o **InsurFlow ERP & AI**).
- 
- **Nombre Técnico para Documentación:** *Sistema Distribuido de Gestión Empresarial (ERP), Peritaje Inteligente y Liquidación Autónoma de Siniestros de Seguros*.
- 
- **Metodología de Desarrollo: ICONIX** (Modelado de Dominio, Casos de Uso con Prototipos, Análisis de Robustez con objetos *Boundary, Control y Entity*, y Diagramas de Secuencia en UML 2.5+).
- 
- **Fecha objetivo de entrega final:** 10 - 12 de noviembre (desarrollo iterativo en 4 semanas).
- 

# 1. Arquitectura en 4 Niveles y Mapeo del Stack Tecnológico

A continuación, te presento el diagrama arquitectónico global de **AegisCore AI** y dónde encaja cada tecnología impuesta:

```
graph TB
    subgraph Nivel_1["Nivel 1: Micro-Frontends, Clientes e IA Local (SSO Keycloak)"]
        SSO["🔐 Keycloak SSO (OAuth2 / OIDC)"]
        MF1["🅰️ Angular (Portal ERP Admin & Dashboards BI)"]
        MF2["⚛️ React (Portal Asegurados + IA Local Web ONNX/TF.js)"]
        MF3["📱 Flutter / PWA (App Peritos Offline + SQLite + IA Local TFLite)"]
    end
```

```
    subgraph Nivel_4["Nivel 4: Gateway, Orquestación y Automatización"]
        GQL["🚀 Apollo GraphQL Router / API Gateway"]
        N8N["⚡ n8n (Automatización WhatsApp -> ERP -> Facturación / Siniestros)"]
        KAFKA["📨 Event Bus (Kafka / RabbitMQ) & Orquestador Saga"]
    end

    subgraph Nivel_2["Nivel 2: Microservicios Backend (4 Lenguajes y 3 Arquitecturas)"]
        MS1["🐘 MS1: Pólizas, Clientes y Facturación<br/>(PHP 8.3 / Laravel)<br/>🏛️ Arquitectura: DDD"]
        MS2["🦁 MS2: Reclamos, Triaje y Siniestros<br/>(TypeScript / NestJS)<br/>🏛️ Arquitectura: Clean Architecture + CQRS"]
        MS3["🍃 MS3: Motor Actuarial, Reservas y Pagos<br/>(Java / Spring Boot 3)<br/>🏛️ Arquitectura: Hexagonal (Ports & Adapters)"]
        MS4["🐍 MS4: IA, Deep Learning, Agentes y MCP Server<br/>(Python / FastAPI + TensorFlow + LangGraph)"]
    end

    subgraph Nivel_3["Nivel 3: Persistencia Políglota (3 Motores de BD)"]
        DB1[("🐘 PostgreSQL<br/>(Relacional: Pólizas, Facturas, Contabilidad)")]
        DB2[("🍃 MongoDB<br/>(Documental: Expedientes de Siniestros, Peritajes)")]
        DB3[("⚡ Amazon DynamoDB<br/>(Clave-Valor: Idempotency-Keys, Sesiones, Caché)")]
    end

    SSO -.-> MF1 & MF2 & MF3
    MF1 & MF2 & MF3 -->|GraphQL / REST + Idempotency-Key| GQL
    N8N -->|Webhooks / GraphQL| GQL
    GQL --> MS1 & MS2 & MS3 & MS4
    MS1 <--> KAFKA
    MS2 <--> KAFKA
    MS3 <--> KAFKA
    MS4 <-->|Protocolo MCP| MS1 & MS2 & MS3
    MS1 & MS3 --> DB1
    MS2 & MS4 --> DB2
    MS1 & MS2 & MS3 --> DB3
```

# Nivel 1: Frontend Heterogéneo (3 Frameworks) + Inicio de Sesión Único (SSO)

1. **Mecanismo de Acceso Único (SSO):** Implementado con **Keycloak** (u OAuth 2.0 / OpenID Connect con JWT). El usuario inicia sesión una sola vez en una entrada unificada y puede cambiar entre el ERP Administrativo, el Portal de Clientes y el módulo de Reportes sin volver a autenticarse.

2. **Micro-Frontend 1 — Angular (Panel ERP Administrativo y BI):** Utilizado por administradores, ajustadores de siniestros, auditores médicos y gerentes. Contiene la gestión empresarial pesada, el **Dashboard estadístico nativo con KPIs** en código (sin Power BI) y la interfaz de **Reportes Generativos por Voz**.

3. **Micro-Frontend 2 — React (Portal de Autoservicio y Cotización Comercial):** Utilizado por clientes/asegurados y corredores de seguros para cotizar pólizas de salud o vehículos, comprar en línea y consultar el estado de sus reclamos mediante **GraphQL**. Incluye **IA Local en el navegador** (usando TensorFlow.js / ONNX Runtime Web) para validar sin internet que las fotos de documentos o vehículos subidas por el cliente no estén borrosas antes de enviarlas.

4. **Cliente 3 — Flutter / PWA (App Móvil para Peritos de Campo):** Aprovecha el hardware del teléfono (cámara, GPS, micrófono) para los inspectores de siniestros. Opera **100% Offline** almacenando las inspecciones en **SQLite** local y ejecuta **IA Local en el dispositivo móvil (TensorFlow Lite)** para detectar partes dañadas del vehículo cuando no hay señal de internet.

## Nivel 2: Microservicios Backend (4 Microservicios, 4 Lenguajes, 3 Arquitecturas de Código)

Para cumplir con la restricción de mínimo 3 microservicios en Spring Boot, NestJS y PHP, más el soporte de IA en Python, dividimos el sistema en 4 microservicios claramente delimitados:

| Microservicio | Lenguaje / Framework | Patrón de Arquitectura de Código | Responsabilidad en el Dominio de Seguros |
|---|---|---|---|
| **MS1: Core ERP de Pólizas y Facturación** | **PHP 8.3** (Laravel) | **DDD (Domain-Driven Design)** | Gestión de clientes/asegurados, catálogo de seguros (vehículos/salud), inscripción/emisión de pólizas, control de coberturas y emisión de **facturas / notas de venta**. Expone API **GraphQL**. |

| | | | |
|---|---|---|---|
| **MS2: Gestión de Siniestros y Triaje** | **TypeScript** (NestJS) | **Clean Architecture** (+ CQRS) | Recepción de reclamos viales y médicos, gestión de expedientes multimedia de siniestros, sincronización offline idempotente y notificaciones Push/WebSockets. Expone API **GraphQL**. |
| **MS3: Motor Actuarial, Cobranzas y Liquidación** | **Java** (Spring Boot 3) | **Arquitectura Hexagonal** (Puertos y Adaptadores) | Cálculo actuarial de primas y deducibles, reserva técnica de fondos, cobro de mensualidades, aprobación de pagos de indemnización y **Orquestador del Patrón Saga**. |
| **MS4: Inteligencia Artificial, Agentes y MCP** | **Python** (FastAPI) | **Arquitectura Orientada a Servicios / Pipelines IA** | Entrenamiento e inferencia de **Machine Learning** (Scikit-Learn) y **Deep Learning** (TensorFlow), **Agentes Inteligentes** autónomos y **Servidor MCP** para reportes por voz. |

## Nivel 3: Persistencia Políglota (3 Gestores de Bases de Datos)

1. **PostgreSQL (Relacional SQL):** Conectado a **MS1 (PHP)** y **MS3 (Spring Boot)**. Almacena datos estructurados que exigen integridad transaccional ACID: usuarios, clientes, pólizas contratadas, coberturas, pagos de primas, facturas/notas de venta y asientos de liquidación.
2. 
3. **MongoDB (NoSQL Documental):** Conectado a **MS2 (NestJS)** y **MS4 (Python)**. Almacena los expedientes dinámicos de siniestros (informes del perito, JSON con coordenadas de abolladuras detectadas por TensorFlow, recetas/diagnósticos médicos escaneados, partes policiales y logs de auditoría de agentes).
4. 

5. **Amazon DynamoDB (NoSQL Clave-Valor):** Conectado transversalmente para operaciones de latencia sub-milisegundo. Almacena la tabla de **Llaves de Idempotencia (**Idempotency-Keys**)** para evitar transacciones duplicadas cuando la app móvil sincroniza con mala señal, además de caché rápida de estado de pólizas y sesiones.
6.

# 2. ¿Dónde y Cómo Implementaremos Cada Tecnología Exigida?

Aquí tienes la respuesta exacta que debes defender en tu presentación preliminar ante el docente:

## A. Protocolo MCP (Model Context Protocol) + Prevención de Inyección SQL por Voz

- **¿Dónde se implementa?:** En el **MS4 (Python / FastAPI)** actuando como **MCP Server** y consumido por el módulo de Reportes Inteligentes y Agentes.
-
- **¿Cómo funciona?:** En lugar de conectar un LLM directamente a la base de datos PostgreSQL (lo cual sería vulnerable a que un usuario diga por micrófono: *"Muéstrame las pólizas; DROP TABLE polizas;"*), crearemos un **Servidor MCP propio** que expone únicamente **Tools (Herramientas) seguras y tipadas**:
-
    - tool_consultar_siniestralidad(fecha_inicio, fecha_fin, tipo_seguro).
    -
    - tool_obtener_polizas_por_estado(estado, limite).
    -
    - tool_exportar_reporte_financiero(formato, trimestre).
    -
- **Defensa de Seguridad OWASP:** El audio se convierte a texto con Whisper; un filtro sanitiza el prompt contra *Prompt Injection*, y el LLM solo puede elegir qué herramienta MCP llamar llenando un esquema JSON estricto. El servidor MCP ejecuta consultas mediante ORMs parametrizados (nunca concatenando SQL dinámico), neutralizando al 100% cualquier intento de **Inyección SQL mediante voz**.
-

## B. Automatización con n8n (Pedidos/Pólizas y Siniestros por WhatsApp)

Implementaremos un flujo automatizado en **n8n** que conecta **3 canales externos/internos (WhatsApp + Core ERP + Facturación/Correo)** cumpliendo exactamente lo pedido en tus apuntes:

1. **Caso 1 (Contratación / Pedido de Seguro y Facturación por WhatsApp):**

    - Un cliente escribe al bot de WhatsApp solicitando adquirir o renovar un seguro vehicular (envía foto de su RUAT/carnet).
    - 
    - **n8n** captura el webhook de WhatsApp, extrae los datos y envía una mutación **GraphQL al MS1 (PHP)** para registrar la orden de póliza.
    - 
    - **n8n** invoca el servicio de facturación en **MS1/MS3** para generar automáticamente la **Factura / Nota de Venta en PDF** y la envía de vuelta al cliente por **WhatsApp y Correo Electrónico**.
    - 
2. **Caso 2 (Reporte de Siniestro Vial de Emergencia por WhatsApp):**
    - El asegurado sufre un choque y manda fotos + ubicación GPS por WhatsApp.
    - 
    - **n8n** recibe las fotos, llama al **MS4 (Python TensorFlow)** para evaluar la gravedad del daño en la imagen, crea el expediente de siniestro en **MS2 (NestJS / MongoDB)** y notifica al perito más cercano y al correo del asegurado.
    - 

# C. Transacciones Transversales en 3 Microservicios (Patrón Saga) e Idempotencia

- **¿Dónde se aplica el Patrón Saga?:** En el proceso crítico de **"Liquidación y Pago de Indemnización de un Siniestro"** (CU07), donde intervienen obligatoriamente **3 microservicios**:
    - **Paso 1 — MS2 (NestJS / MongoDB):** Cambia el estado del reclamo a `APROBADO_PARA_LIQUIDACION`.
    - 
    - **Paso 2 — MS1 (PHP / PostgreSQL):** Verifica la póliza activa y descuenta el monto indemnizado (ej. $2,500 USD) del techo de cobertura anual del asegurado.
    - 
    - **Paso 3 — MS3 (Spring Boot / PostgreSQL):** Reserva los fondos actuariales, genera el asiento contable y emite la orden de pago/factura de liquidación.
    - 
- **¿Qué ocurre si uno de los microservicios falla? (Compensación / Rollback):**
    - Si el **Paso 3 (Spring Boot)** falla (por ejemplo, fondos insuficientes en la reserva técnica, caída de base de datos o cuenta bancaria rechazada), el **Orquestador Saga** ejecuta transacciones compensatorias en orden inverso:
    - 

        - Dispara `CompensarCoberturaCommand` hacia **MS1 (PHP)** para devolver los $2,500 USD al saldo de la póliza del cliente.
        -
        - Dispara `RevertirEstadoReclamoCommand` hacia **MS2 (NestJS)** para devolver el siniestro a estado `OBSERVADO_FINANZAS` y notificar al ajustador.
        -
- **¿Cómo se resuelve la Idempotencia?:**

    - En zonas con pésima señal de internet, la App Móvil (**Flutter**) o el Front Web pueden reintentar enviar la misma solicitud de creación de siniestro o pago 2 o 3 veces.
    - Cada petición incluye en el header un `Idempotency-Key` único (UUIDv4 generado en el cliente antes de guardar en SQLite).
    - Antes de procesar la transacción, el backend consulta **Amazon DynamoDB**. Si la clave ya existe, **no vuelve a ejecutar la lógica ni duplica el cobro/siniestro**, sino que retorna inmediatamente el resultado JSON que quedó guardado en el primer intento exitoso.

# D. Machine Learning (Supervisado y No Supervisado) y Deep Learning (TensorFlow)

- **Machine Learning Supervisado (Scikit-Learn — Random Forest):**

    - *Aplicación:* **Predicción de Siniestralidad y Prima Óptima / Predicción de Ventas de Pólizas** usando un dataset histórico (o sintético de varios meses). Permite predecir cuánto costará un reclamo o cuántas pólizas se venderán el próximo mes en el Dashboard.
- **Machine Learning No Supervisado (Scikit-Learn — K-Means + Isolation Forest):**

    - *Aplicación 1 (K-Means):* **Segmentación de Asegurados** en clústeres según perfil de riesgo, frecuencia de reclamos y monto de prima pagada (Clientes Bajo Riesgo, Riesgo Moderado, Alta Siniestralidad).
    - *Aplicación 2 (Isolation Forest):* **Detección de Anomalías y Fraude** en reclamos médicos o vehiculares que se salen drásticamente del patrón normal (ej. cobros duplicados o montos inflados).
- **Deep Learning con TensorFlow (Visión Artificial - Imagen y Video):**

- *Aplicación:* En **MS4 (Python + TensorFlow/Keras)**, un modelo de redes neuronales convolucionales (CNN / MobileNet / YOLO sobre TensorFlow) analiza las **fotografías o videos cortos de vehículos chocados** y facturas médicas para clasificar automáticamente las piezas dañadas (parachoques, farol, capó) y estimar el porcentaje de severidad del daño.

## E. Agentes Inteligentes Funcionales (En Nube y Locales / Offline)

1. **Agente Inteligente en Dispositivo Móvil y Web (IA Local / Offline):**
   - *Misión del Agente:* Asistir al perito en terreno durante una inspección sin conexión a internet.
   - *Funcionamiento:* Utiliza un modelo ligero exportado a **TensorFlow Lite (en Flutter)** y **TensorFlow.js / ONNX (en React Web)**. Verifica localmente que las fotos tomadas correspondan a un vehículo, extrae la placa por OCR offline y guía al perito sobre qué ángulos fotográficos le faltan capturar antes de guardar todo en **SQLite**.
2. **Agente Autónomo de Monitoreo en el Backend (Metas y Skills):**
   - *Misión del Agente:* Monitorear continuamente la cola de siniestros y las métricas financieras sin intervención humana.
   - *Skills (Habilidades):* Si detecta una anomalía de fraude (vía Isolation Forest), congela preventivamente la liquidación automática y envía una alerta al auditor; si detecta un siniestro grave sin perito asignado por más de 15 minutos, reasigna automáticamente al inspector más cercano por GPS.

## F. Reportes Generativos por Voz en Lenguaje Natural

- **¿Dónde se ubica?:** En el **Micro-Frontend de Angular** conectado al **MS4 (Python + Whisper + MCP)**.
- **Flujo completo:** El gerente presiona el botón de micrófono y dice: *"Genera un reporte de las pólizas vendidas y siniestros liquidados del último trimestre agrupados por tipo de vehículo"*.
- El sistema transcribe la voz a texto, consulta los datos mediante el **Protocolo MCP**, renderiza **gráficos dinámicos en pantalla** y habilita la descarga instantánea en **Excel (.xlsx), PDF y Word (.docx)**.

# 3. Estructura de Entregables bajo Metodología ICONIX

Para cumplir con la metodología **ICONIX** exigida en tus apuntes y con el estándar **UML 2.5+** que pide el docente, estructuraremos tu documento preliminar en las siguientes 4 fases:

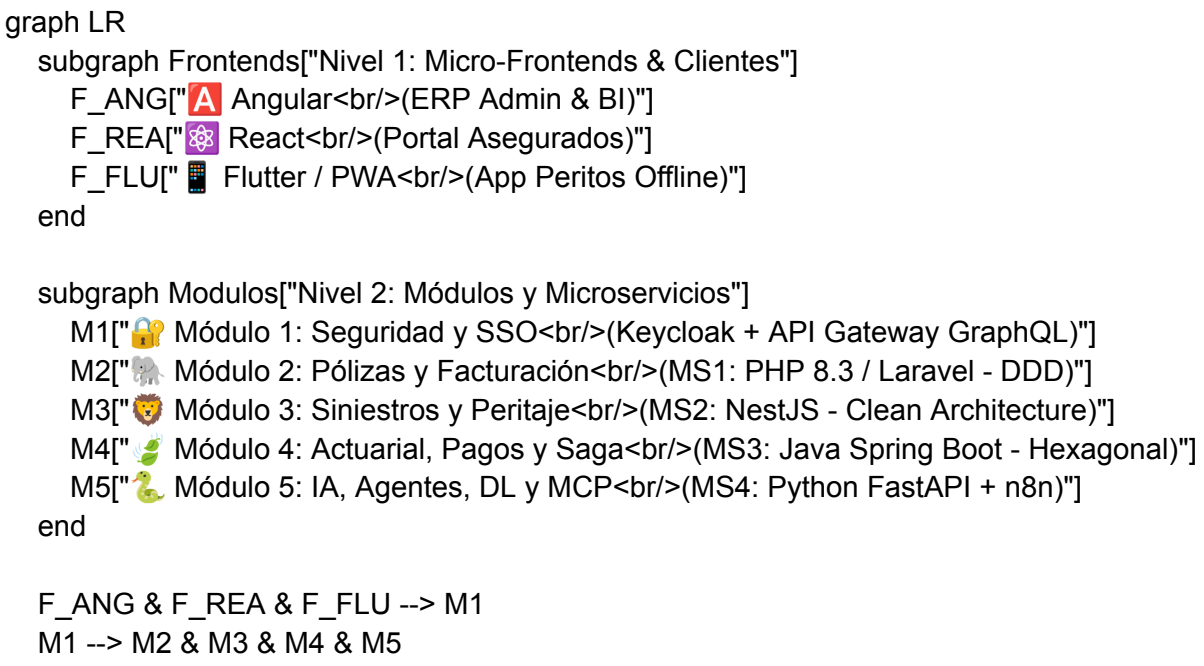
1. **Análisis de Requisitos:** Modelo de Dominio (entidades principales), Lista de Requisitos Funcionales/No Funcionales y Prototipos rápidos de interfaz (GUI).

2. **Análisis y Diseño Preliminar:** Casos de Uso detallados y **Diagramas de Robustez** (conectando explícitamente los objetos *Interfaz/Boundary*, *Control* y *Entidad/Entity* de cada caso de uso).

3. **Diseño Detallado: Diagramas de Secuencia (UML 2.5+)** reflejando los métodos reales de las clases de control, el flujo GraphQL y la compensación Saga, junto con el Diseño Físico de las 3 Bases de Datos.

4. **Implementación y Pruebas:** Despliegue en contenedores Docker/Kubernetes y validación de casos de prueba.

# Mapa General de Módulos y Distribución Arquitectónica




## Catálogo Detallado de Casos de Uso por Módulo

### 🔐 MÓDULO 1: Identidad, Seguridad Perimetral y Federación de Datos (Transversal)

Este módulo actúa como el punto de entrada unificado (Nivel 1 y Nivel 4), gestionando la sesión única y la agregación de consultas para los tres clientes frontend.

#### CU01: Iniciar Sesión Único Multi-Aplicación (SSO - Single Sign-On)

- **Actores:** Cliente Asegurado, Perito de Campo, Ajustador Administrativo, Gerente de Operaciones.
- **Descripción Técnica (ICONIX):**
    1. **Boundary:** LoginUnificadoSSOView (Shell Container Web/Móvil).
    2. **Control:** AuthOIDCController (Keycloak OAuth 2.0 / OpenID Connect + PKCE).
    3. **Entity:** SesionUnificadaToken (en **Amazon DynamoDB**) y UsuarioRol (en **PostgreSQL**).
    4. **Flujo:** El usuario se autentica una única vez en la puerta de enlace central y recibe un token JWT firmado (RS256) que le permite cambiar entre el **Portal Administrativo (Angular)**, el **Portal de Clientes (React)** y la **App de Peritos (Flutter)** sin volver a iniciar sesión.
- **Tecnologías Base:** Keycloak, JWT, Angular, React, Flutter, Amazon DynamoDB.
- 🎯 **¿Aplica Tecnología Impuesta?: SÍ — OBLIGATORIA:**
    1. **Mecanismo de Inicio de Sesión Único (SSO):** Ingreso por una sola entrada sin complicarse al cambiar entre aplicaciones del ecosistema.
    2. **Integración de los 3 Micro-Frontends:** Conecta Angular, React y el cliente móvil/PWA bajo un mismo contexto de identidad.

**CU02: Consultar Estado Unificado de Pólizas, Vehículos y Siniestros (Federación GraphQL)**

- **Actores:** Asegurado / Corredor de Seguros (vía Portal Web React o App Móvil).
- **Descripción Técnica (ICONIX):**
    1. **Boundary:** ResumenCoberturaClienteView (React + Apollo Client).
    2. **Control:** ApolloFederationResolver (Gateway GraphQL conectando MS1, MS2 y MS3).
    3. **Entity:** Poliza (PostgreSQL), ExpedienteSiniestro (MongoDB), CacheCoberturaActiva (DynamoDB).
    4. **Flujo:** En una sola petición GraphQL con subselección de campos, el frontend obtiene los datos del titular, sus vehículos o beneficiarios asegurados, su saldo de cobertura anual y el estado de sus reclamos activos sin hacer múltiples llamadas REST.
- **Tecnologías Base:** React, Apollo GraphQL Router, PHP (MS1), NestJS (MS2), PostgreSQL, MongoDB, DynamoDB.
- 🎯 **¿Aplica Tecnología Impuesta?: SÍ — OBLIGATORIA:**
    1. **Comunicación mediante GraphQL en el Sistema de Gestión Empresarial:** Aplicado directamente a la lógica de negocio e inscripciones de pólizas.
    2. **Arquitectura Nivel 3 (Agregación y Caching de Datos):** Consolida datos dispersos en los 3 motores de bases de datos (PostgreSQL, MongoDB y DynamoDB).

🐘 **MÓDULO 2: Gestión Comercial, Pólizas y Facturación — Core ERP (MS1)**

Representa el corazón administrativo del negocio asegurador (contratos, inscripciones de asegurados, catálogo de coberturas y facturación fiscal).

- **Stack del Módulo:** Backend en **PHP 8.3 (Laravel)** | Patrón de Código: **DDD (Domain-Driven Design)** | Base de Datos: **PostgreSQL** | API: **GraphQL**.

**CU03: Gestionar Catálogo de Planes de Seguro, Coberturas y Clientes (Core ERP)**

- **Actores:** Administrador del ERP.
- **Descripción Técnica (ICONIX):**
    1. **Boundary:** GestionCatalogoSegurosView (Micro-Frontend **Angular**).
    2. **Control:** CatalogoPolizaApplicationService (PHP - Agregados y Servicios de Dominio DDD).
    3. **Entity:** Cliente, PlanSeguro, Cobertura, BienAsegurado (Tablas transaccionales en **PostgreSQL**).
    4. **Flujo:** Administración completa (CRUD vía mutaciones y queries GraphQL) de los planes de seguro (Todo Riesgo Vehicular, Responsabilidad Civil, Seguro Médico), topes de cobertura, deducibles y registro de clientes.
- **Tecnologías Base:** Angular, GraphQL, PHP 8.3, PostgreSQL.
- 🎯 **¿Aplica Tecnología Impuesta?: SÍ — OBLIGATORIA:**
    1. **Lenguaje Backend PHP + Base de Datos Relacional PostgreSQL**.
    2. **Patrón de Arquitectura a Nivel de Código #1 — DDD (Domain-Driven Design):** Modelado mediante Entidades Raíz (*Aggregate Roots*), Objetos de Valor (*Value Objects* como PrimaMonetaria, VigenciaPoliza) y Repositorios de Dominio.

**CU04: Cotizar y Emitir Póliza en Línea con Validación de Documentos por IA Local Web**

- **Actores:** Cliente / Asegurado.
- **Descripción Técnica (ICONIX):**
    1. **Boundary:** CotizadorInscripcionWebView (Micro-Frontend **React** con PWA Offline y modelo **ONNX Runtime Web / TensorFlow.js**).
    2. **Control:** ValidadorNitidezLocalWebControl (en navegador) y EmisionPolizaGraphQLController (en **PHP**).
    3. **Entity:** Cotizacion, ContratoPoliza (en **PostgreSQL**).
    4. **Flujo:** El cliente cotiza su póliza en React y sube fotografías de su documento de identidad, RUAT o estado actual del vehículo. Antes de enviar al servidor (incluso sin internet), un modelo de **IA Local en el navegador web** valida la nitidez de la imagen y que el documento sea legible. Al confirmar, ejecuta una mutación GraphQL que emite la póliza en PostgreSQL.
- **Tecnologías Base:** React, TensorFlow.js / ONNX Web, Service Workers (PWA Offline), GraphQL, PHP, PostgreSQL.
- 🎯 **¿Aplica Tecnología Impuesta?: SÍ — OBLIGATORIA:**
    1. **Front Web con IA Local y Funcionamiento Offline:** Inferencia ejecutada directamente en el navegador del cliente sin depender del servidor.
    2. **Lógica de Inscripción/Emisión con GraphQL en Frontend React**.

**CU05: Contratar / Renovar Póliza por WhatsApp y Emitir Factura Automática (Flujo n8n)**

- **Actores:** Cliente (vía WhatsApp), Orquestador **n8n**.
- **Descripción Técnica (ICONIX):**
    1. **Boundary:** WhatsAppWebhookBoundary (Canal WhatsApp Business API conectado a **n8n**).
    2. **Control:** WorkflowPedidoPolizaN8N (Orquestador n8n), RecepcionPedidoService (PHP) y FacturacionService (PHP/Spring Boot).
    3. **Entity:** OrdenSuscripcionPoliza, FacturaNotaVenta (en **PostgreSQL**).

4. **Flujo:** El cliente solicita comprar o renovar un seguro por WhatsApp enviando sus datos. El flujo en **n8n** toma el pedido de WhatsApp, lo registra en el sistema de gestión empresarial (**MS1 PHP**), invoca automáticamente al proceso de facturación para generar la **Factura / Nota de Venta en PDF** y se la envía al cliente por WhatsApp y Correo Electrónico.

- **Tecnologías Base:** n8n, WhatsApp API, SMTP (Correo), GraphQL/REST, PHP, PostgreSQL.
- 🎯 **¿Aplica Tecnología Impuesta?: SÍ — OBLIGATORIA:**
  1. **Automatización con n8n (Pedidos por WhatsApp → ERP → Factura/Nota de Venta):** Cumple al pie de la letra la nota manuscrita y el apunte de clase sobre tomar pedidos por WhatsApp, recepcionarlos en el ERP y pasar a otro servicio para generar la factura o nota de venta.

### 🦁 MÓDULO 3: Gestión de Siniestros, Peritaje Móvil Offline y Triaje (MS2)

Gestiona el ciclo de vida de los reclamos (accidentes viales y emergencias médicas), la inspección pericial en campo sin internet y la sincronización tolerante a fallos de red.

- **Stack del Módulo:** Backend en **TypeScript (NestJS 10)** | Patrón de Código: **Clean Architecture (+ CQRS)** | Base de Datos: **MongoDB** + **DynamoDB** | Cliente Móvil: **Flutter**.

#### CU06: Reportar Siniestro Vial de Emergencia por WhatsApp con Triaje IA (Flujo n8n)

- **Actores:** Asegurado Accidentado, Plataforma **n8n**, Perito / Ajustador.
- **Descripción Técnica (ICONIX):**
  1. **Boundary:** WhatsAppSiniestroBotBoundary.
  2. **Control:** FlujoEmergenciaN8N, RegistrarSiniestroUseCase (**NestJS** - Clean Architecture) y EvaluarSeveridadImagenService (**Python**).
  3. **Entity:** ExpedienteSiniestroDocument (en **MongoDB**).
  4. **Flujo:** El asegurado envía fotos del choque y su ubicación GPS por WhatsApp. **n8n** recibe las evidencias, las pasa por el microservicio de IA en Python para clasificar la gravedad inicial, crea el expediente documental en **MongoDB** a través de **NestJS** y envía una **Notificación Push** en tiempo real a la app móvil del perito más cercano.
- **Tecnologías Base:** n8n, NestJS, Python, MongoDB, WebSockets / Push Notifications (FCM).
- 🎯 **¿Aplica Tecnología Impuesta?: SÍ — OBLIGATORIA:**
  1. **Lenguaje Backend NestJS + Base de Datos Documental MongoDB**.
  2. **Patrón de Arquitectura a Nivel de Código #2 — Clean Architecture:** Casos de uso concéntricos independientes de frameworks y separación de comandos/consultas (CQRS).
  3. **Notificaciones Push en tiempo real** hacia los dispositivos móviles.

#### CU07: Inspeccionar Daños del Vehículo en Terreno sin Internet con Agente de IA Local

- **Actores:** Perito / Inspector de Campo.
- **Descripción Técnica (ICONIX):**
  1. **Boundary:** CamaraInspeccionPericialView (App Móvil **Flutter** usando hardware nativo: Cámara, GPS y Micrófono).

2. **Control:** AgentePericialLocalMobileControl (Agente funcional local con **TensorFlow Lite / LiteRT** embebido en el móvil).
3. **Entity:** InspeccionOfflineSQLiteEntity (Base de datos local **SQLite** en el teléfono).
4. **Flujo:** En una carretera o sótano sin cobertura de internet, el perito toma fotos del vehículo siniestrado. El **Agente Funcional de IA Local en el móvil** tiene la misión de asistir la inspección: ejecuta visión artificial offline para detectar partes abolladas, extrae la placa por OCR local y verifica que se hayan tomado los 4 ángulos reglamentarios. Todo el peritaje y el Idempotency-Key generado se guardan localmente en **SQLite**.

- **Tecnologías Base:** Flutter, TensorFlow Lite (Edge AI), SQLite, Sensores Hardware (GPS, Cámara).
- 🎯 **¿Aplica Tecnología Impuesta?: SÍ — OBLIGATORIA:**
  1. **Tercer Frontend / Cliente Móvil (Flutter/PWA) con Funcionamiento Offline (SQLite)**.
  2. **Agente Funcional con IA Local en el Dispositivo Móvil:** Cumple el requisito de darle una misión al agente ejecutando IA directamente en el teléfono sin internet.

#### CU08: Sincronizar Expediente de Siniestro con Control de Idempotencia

- **Actores:** App Móvil Flutter (Proceso de Sincronización en Segundo Plano), Backend **NestJS**.
- **Descripción Técnica (ICONIX):**
  1. **Boundary:** SyncBackgroundServiceWorker (Flutter).
  2. **Control:** IdempotencyGuardInterceptor y SincronizarPeritajeUseCase (**NestJS**).
  3. **Entity:** IdempotencyRecord (en **Amazon DynamoDB**) y ExpedienteSiniestroDocument (en **MongoDB**).
  4. **Flujo:** Cuando el perito recupera una señal de internet inestable, la app intenta subir el expediente enviando en la cabecera un Idempotency-Key (UUIDv4 + Hash del payload). Si por micro-cortes de red el teléfono envía la petición 3 veces, **NestJS** consulta en sub-milisegundos la tabla en **Amazon DynamoDB**: al detectar que la llave ya fue procesada en el primer intento, bloquea las ejecuciones repetidas y devuelve la respuesta almacenada, evitando duplicar el siniestro en **MongoDB**.
- **Tecnologías Base:** Flutter, NestJS, Amazon DynamoDB, MongoDB.
- 🎯 **¿Aplica Tecnología Impuesta?: SÍ — OBLIGATORIA:**
  1. **Mecanismo de Idempotencia y Sincronización con Pésima Señal de Internet:** Resuelve explícitamente el problema de no registrar transacciones dos veces.
  2. **Tercer Motor de Base de Datos (NoSQL Clave-Valor — Amazon DynamoDB)**.

### 🍃 MÓDULO 4: Motor Actuarial, Cobranzas y Liquidación Financiera (MS3)

Responsable de los cálculos actuariales de precisión, cobro de mensualidades, reservas técnicas y la coordinación de transacciones distribuidas entre microservicios.

- **Stack del Módulo:** Backend en **Java (Spring Boot 3)** | Patrón de Código: **Arquitectura Hexagonal (Ports & Adapters)** | Base de Datos: **PostgreSQL** + **DynamoDB** | Broker: **Apache Kafka**.

**CU09: Liquidar y Pagar Indemnización de Siniestro (Transacción Transversal Saga en 3 Microservicios)**

- **Actores:** Ajustador de Seguros / Gerente Financiero (vía **Angular**).
- **Descripción Técnica (ICONIX):**
    1. **Boundary:** LiquidacionSiniestroAdminView (**Angular**).
    2. **Control:** SagaLiquidacionOrchestrator (**Spring Boot** - Puerto de Entrada Hexagonal), PolizaCoberturaService (**PHP**) y ReclamoEstadoUseCase (**NestJS**).
    3. **Entity:** OrdenLiquidacionPago (PostgreSQL - MS3), ContratoPoliza (PostgreSQL - MS1) y ExpedienteSiniestro (MongoDB - MS2).
    4. **Flujo Normal (3 Microservicios):** Al aprobar una indemnización de $3,000 USD, el Orquestador Saga ejecuta en secuencia:
        1. **MS2 (NestJS / MongoDB):** Cambia el estado del expediente a EN_PROCESO_LIQUIDACION.
        2. **MS1 (PHP / PostgreSQL):** Verifica la vigencia y descuenta $3,000 USD del techo anual de cobertura de la póliza.
        3. **MS3 (Spring Boot / PostgreSQL):** Descuenta la reserva matemática actuarial, aplica el deducible y genera el desembolso bancario y comprobante contable.
    5. **Flujo de Compensación (Rollback Semántico ante Fallo):** Si el Paso 3 en **Spring Boot** falla (ej. cuenta bancaria bloqueada o saldo de reserva insuficiente), el Orquestador Saga dispara eventos de compensación en **Kafka**: ordena a **MS1 (PHP)** restituir los $3,000 USD al saldo de la póliza y ordena a **MS2 (NestJS)** revertir el estado del siniestro a PENDIENTE_REVISION_FINANCIERA.
- **Tecnologías Base:** Angular, Java Spring Boot 3, PHP 8.3, NestJS, Apache Kafka, PostgreSQL, MongoDB, DynamoDB.
- 🎯 **¿Aplica Tecnología Impuesta?: SÍ — OBLIGATORIA (NÚCLEO DE DEFENSA):**
    1. **Transacción Transversal Involucrando 3 Microservicios con Patrón Saga:** Demuestra paso a paso qué ocurre cuando uno de los microservicios falla y cómo se ejecuta la compensación.
    2. **Lenguaje Backend Java (Spring Boot) + Patrón de Arquitectura a Nivel de Código #3 — Arquitectura Hexagonal (Ports & Adapters)**.

**CU10: Procesar Cobro Recurrente de Primas y Pago por QR con Control Idempotente**

- **Actores:** Asegurado (App Móvil / Web React) o Scheduler Financiero.
- **Descripción Técnica (ICONIX):**
    1. **Boundary:** PasarelaPagoPrimaQRView (React / Flutter).
    2. **Control:** CobranzaPrimaUseCasePort (**Java Spring Boot** - Arquitectura Hexagonal).
    3. **Entity:** PagoPrimaTransaccion (en **PostgreSQL**) y IdempotencyLockPago (en **DynamoDB**).

4. **Flujo:** El asegurado paga su cuota mensual mediante código QR o tarjeta. **Spring Boot** verifica la clave de idempotencia en **DynamoDB** para impedir cobros dobles, registra el ingreso contable en **PostgreSQL** y emite el evento de renovación de vigencia hacia **MS1 (PHP)**.

- **Tecnologías Base:** React/Flutter, Java Spring Boot 3, PostgreSQL, Amazon DynamoDB.
- 🎯 **¿Aplica Tecnología Impuesta?: SÍ — OBLIGATORIA:**
    1. **Aplicación conjunta de Arquitectura Hexagonal en Spring Boot + Idempotencia Financiera en DynamoDB**.

### 🐍 MÓDULO 5: Inteligencia Artificial, Deep Learning, Agentes Autónomos y BI por Voz (MS4)

Concentra los modelos predictivos supervisados, de clustering/anomalías no supervisados, visión artificial profunda con TensorFlow, agentes autónomos y el servidor MCP para reportes generativos por voz.

- **Stack del Módulo:** Backend en **Python (FastAPI)** | Librerías IA: **Scikit-Learn, TensorFlow/Keras, Whisper, LangGraph, MCP SDK** | Bases de Datos: **MongoDB** + Réplicas de Lectura **PostgreSQL**.

#### CU11: Analizar Fotografía/Video de Siniestro y Facturas Médicas con Deep Learning (TensorFlow)

- **Actores:** Ajustador de Siniestros / Auditor Médico (o invocado automáticamente desde MS2).
- **Descripción Técnica (ICONIX):**
    1. **Boundary:** VisorPeritajeIAView (**Angular**).
    2. **Control:** DeepLearningVisionController (**Python FastAPI + TensorFlow/Keras**).
    3. **Entity:** ResultadoPeritajeRedesNeuronales (JSON guardado en **MongoDB**).
    4. **Flujo:** El microservicio recibe las imágenes o videos cortos del choque vehicular o facturas clínicas. Ejecuta redes neuronales convolucionales en **TensorFlow** (MobileNet / YOLO / Mask R-CNN) para segmentar las piezas dañadas (parachoques, puerta, farol), calcular el porcentaje de daño estructural y estimar el costo de reparación en taller.
- **Tecnologías Base:** Python, FastAPI, TensorFlow / Keras, OpenCV, MongoDB, Angular.
- 🎯 **¿Aplica Tecnología Impuesta?: SÍ — OBLIGATORIA:**
    1. **Deep Learning con TensorFlow orientado al Procesamiento de Imagen y Video**.

#### CU12: Predecir Siniestralidad, Ventas de Pólizas y Prima Óptima (Machine Learning Supervisado)

- **Actores:** Gerente Comercial / Actuario (vía Dashboard **Angular**).
- **Descripción Técnica (ICONIX):**
    1. **Boundary:** DashboardPredictivoBIView (Gráficos nativos en código dentro de **Angular**).
    2. **Control:** RandomForestPredictorService (**Python + Scikit-Learn**).

3. **Entity:** DatasetHistoricoPolizas y PrediccionComercial (en **PostgreSQL / MongoDB**).
4. **Flujo:** Entrenado con un dataset histórico/sintético de varios meses de ventas de seguros y reclamos, el algoritmo supervisado **Random Forest** predice la demanda de pólizas del próximo mes (ej. noviembre/diciembre) y calcula el valor óptimo de la prima anual según la edad del conductor, marca/modelo del vehículo e historial de choques.

- **Tecnologías Base:** Python, Scikit-Learn (Random Forest), Pandas, Angular (Chart.js / ECharts nativo en código).
- 🎯 **¿Aplica Tecnología Impuesta?: SÍ — OBLIGATORIA:**
  1. **Machine Learning Supervisado (Random Forest)** aplicado a la predicción de ventas/primas sobre datos históricos.
  2. **Dashboard Nativo con KPIs Estadísticos en Código (sin Power BI)**.

**CU13: Segmentar Asegurados por Riesgo y Detectar Fraudes (ML No Supervisado)**

- **Actores:** Auditor de Riesgos / Analista BI (vía **Angular**).
- **Descripción Técnica (ICONIX):**
  1. **Boundary:** PanelSegmentacionYAnomaliasView (**Angular**).
  2. **Control:** UnsupervisedClusteringAnomalyService (**Python + Scikit-Learn**).
  3. **Entity:** ClusterAsegurado y AlertaFraudeAnomalia (en **MongoDB**).
  4. **Flujo:** Ejecuta dos modelos de **Machine Learning No Supervisado**:
     1. **K-Means:** Agrupa la cartera de clientes en clústeres sin etiquetas previas según frecuencia de siniestros, antigüedad y monto pagado (ej. *Conductores Prudentes VIP*, *Riesgo Estándar*, *Alta Siniestralidad*).
     2. **Isolation Forest:** Detecta reclamos atípicos o fraudulentos (ej. talleres mecánicos o asegurados que reportan siniestros con costos fuera de distribución normal).
- **Tecnologías Base:** Python, Scikit-Learn (K-Means + Isolation Forest), MongoDB, Angular.
- 🎯 **¿Aplica Tecnología Impuesta?: SÍ — OBLIGATORIA:**
  1. **Machine Learning No Supervisado (Clustering K-Means y Detección de Anomalías Isolation Forest)**.

**CU14: Monitorear Siniestros Estancados y Riesgo de Liquidez con Agentes Autónomos (Goals & Skills)**

- **Actores:** Agente Inteligente Autónomo de Backend (sin intervención humana).
- **Descripción Técnica (ICONIX):**
  1. **Boundary:** EventBusListenerBoundary (Suscriptor de eventos en **Apache Kafka**).
  2. **Control:** AutonomousInsuranceAgent (**Python + LangGraph** configurado con *Goals* y *Skills*).
  3. **Entity:** BitacoraAccionAgente (en **MongoDB**).
  4. **Flujo:** El agente se ejecuta continuamente escuchando eventos del negocio con dos misiones claras:
     - **Meta 1:** Evitar cuellos de botella en siniestros graves. **Skill 1:** Si un siniestro vial lleva más de 15 minutos sin atención, localiza por geocerca GPS al perito libre más cercano, se lo reasigna en **MS2** y le dispara una notificación push.

        - **Meta 2:** Proteger las reservas financieras y frenar fraudes. **Skill 2:** Si Isolation Forest marca una probabilidad de fraude alta, congela automáticamente el pase a liquidación Saga y envía una alerta crítica al gerente.
- **Tecnologías Base:** Python, LangGraph, Apache Kafka, MongoDB.
- 🎯 **¿Aplica Tecnología Impuesta?: SÍ — OBLIGATORIA:**
    1. **Agentes Inteligentes Funcionales Basados en Metas y Habilidades (Goals & Skills):** Monitoreo autónomo y toma de decisiones proactiva dentro del ERP.

**CU15: Generar Reportes Dinámicos por Voz en Lenguaje Natural mediante Protocolo MCP y Blindaje Anti-SQLi**

- **Actores:** Gerente General / Administrador (desde Web **Angular** o App Móvil **Flutter**).
- **Descripción Técnica (ICONIX):**
    1. **Boundary:** ConsolaReporteVozNaturalView (Capturador de micrófono/texto en **Angular** y visor de tablas/gráficos exportables).
    2. **Control:** WhisperSpeechToTextControl, OWASPPromptSanitizerGuard, MCPClientAgent y InsuranceMCPServer (**Python FastAPI**).
    3. **Entity:** Vistas analíticas de Poliza, Factura y Siniestro (Réplicas de lectura en **PostgreSQL** y **MongoDB**).
    4. **Flujo Completo y Defensa contra Inyección SQL por Voz:**
        1. El usuario presiona el micrófono y dicta en lenguaje natural: *"Muéstrame las pólizas vendidas y siniestros liquidados del primero al diez de octubre por tipo de seguro y expórtalo a Excel y PDF"*.
        2. **Whisper** convierte la voz a texto.
        3. **Capa de Seguridad OWASP:** Si un atacante intenta dictar por voz un ataque como *"Lista las pólizas; DROP TABLE polizas;"*, el OWASPPromptSanitizerGuard y la arquitectura del **Protocolo MCP (Model Context Protocol)** impiden que el LLM escriba o concatene código SQL libre. El LLM únicamente puede invocar **Tools MCP predefinidas y tipadas con JSON Schema** (ej. mcp_tool_reporte_siniestralidad(fecha_inicio, fecha_fin, formato)), las cuales ejecutan consultas parametrizadas seguras vía ORM.
        4. El sistema muestra el reporte dinámico y los gráficos en pantalla y genera instantáneamente los archivos descargables en **Excel (.xlsx), PDF, Word (.docx) y CSV**.
- **Tecnologías Base:** Angular, Python, Whisper STT, **Protocolo MCP**, OpenPyXL (Excel), ReportLab (PDF), python-docx (Word), PostgreSQL, MongoDB.
- 🎯 **¿Aplica Tecnología Impuesta?: SÍ — OBLIGATORIA (PUNTO CRÍTICO DE EXAMEN):**
    1. **Reportes Generativos mediante Voz y Texto en Lenguaje Natural** (cero reportes estáticos).
    2. **Visualización en Pantalla y Exportación a Excel, PDF, Word y CSV**.
    3. **Aplicación del Protocolo MCP (Model Context Protocol) + Prevención de Inyecciones SQL mediante Voz (Seguridad OWASP)**.